

Interface Segregation & Dependency Inversion

SWEN1 | BIF3

FH University of Applied Sciences
TECHNIKUM WIEN

Table of Contents

- Interface Segregation Principle
- Dependency Inversion Principle
- Dependency Injection

Interface Segregation Principle



FH University of Applied Sciences
TECHNIKUM WIEN

Interface Segregation Principle

- The ISP was first used and formulated by Robert C. Martin while consulting for Xerox (~2000). (https://en.wikipedia.org/wiki/Interface_segregation_principle)
- *"Clients should not be forced to depend upon interfaces they do not use."* — R.C. Martin (2000)
- AKA: *"Many client specific interfaces are better than one general purpose interface."*

Pain points of a *fat interface*:

- Concretions implement irrelevant methods → `NotImplementedException`
- Changes ripple to all implementors
- Violates SRP



Code smell: Worker (C#, before ISP)

```
public interface IWorker
{
    void Work();
    void Eat();
    void Sleep();
}

public class Robot : IWorker  // ⚠ violates ISP
{
    public void Work() { /* … */ }
    public void Eat()  => throw new NotSupportedException();
    public void Sleep() => throw new NotSupportedException();
}
```

Robot depends on **methods it never** uses.

Refactor: split into micron-interfaces (C#)

```
public interface IWorkable { void Work(); }
public interface IRechargeable { void Recharge(); }   // replaces Sleep
public interface IFeedable { void Eat(); }

public class Robot : IWorkable, IRechargeable
{
    public void Work() { /* … */ }
    public void Recharge() { /* charge battery */ }
}
```

Robot now implements only what it needs.

Code smell: Printer (Java, before ISP)

```java
interface MultifunctionDevice {
    void print(Document d);
    void scan(Document d);
    void fax(Document d);
}

class BasicLaserPrinter implements MultifunctionDevice {
    public void print(Document d) { /* ok */ }
    public void scan(Document d) { /* ok */ }
    public void fax(Document d) { throw new UnsupportedOperationException(); }
}
```

Refactor (Java)

```java
interface Printer { void print(Document d); }
interface Scanner { void scan(Document d); }
interface Fax { void fax(Document d); }

class BasicLaserPrinter implements Printer, Scanner {
    public void print(Document d) { /* … */ }
    public void scan(Document d)  { /* … */ }
}
```

No unused methods, smaller change surface.

ISP - Guidelines & Benefits

- Publish **small, cohesive interfaces**; aggregate if needed.
- Prefer **interface inheritance** over flags (`implements X, Y`) when families grow.
- Improves testability (mock only required behaviour).
- Reduces recompilation & redeployment cost.

Conclusion

- Keep interfaces **small & client-specific** (ISP).
- Summarize multiple small interfaces to combined interfaces.



Further Reading

- Agile software development : principles, patterns and practices
  Martin, Robert Cecil (UAS TechnikumWien Library)
- Learn Interface Segregation Principle in C# (+ Examples)

Dependency Inversion Principle

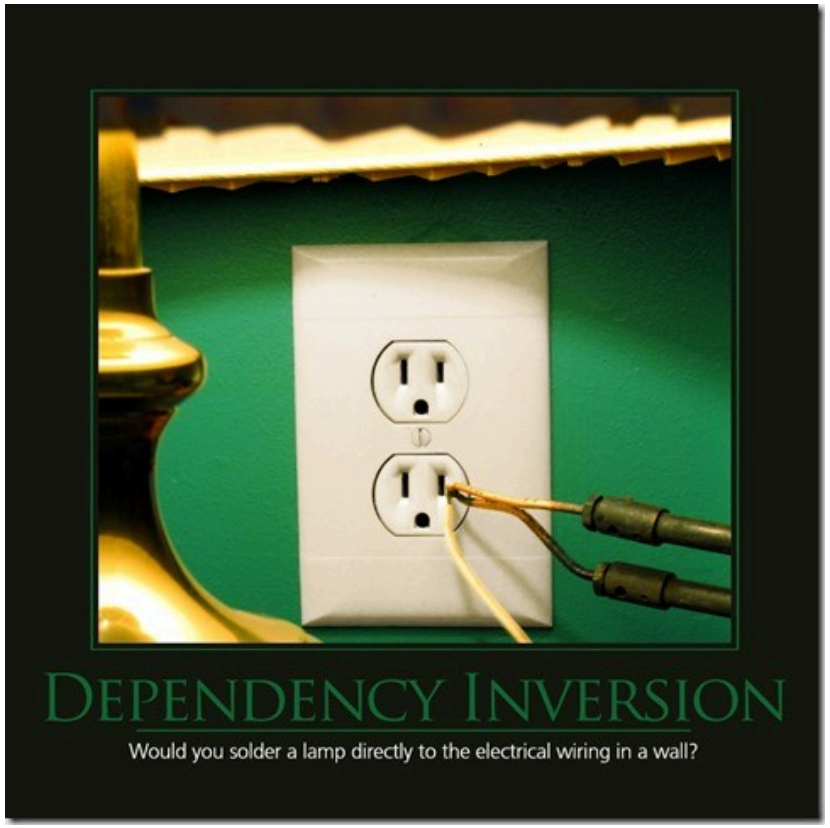


FH University of Applied Sciences
TECHNIKUM WIEN

Dependency Inversion Principle

- Postulated by Robert C. Martin and described in several publications including an article appearing in the C++ Report in May 1996 entitled The Dependency Inversion Principle (https://en.wikipedia.org/wiki/Dependency_inversion_principle)
- *"High-level modules should not depend on low-level modules. Both should depend on abstractions."* — R.C. Martin (1996)
- *"Depend upon Abstractions. Do not depend upon concretions."*

Break the **direction of dependency**—let policies depend on interfaces.



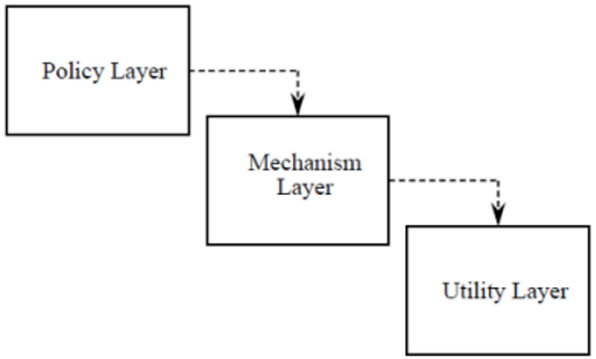
Dependency Inversion Principle

- Super-Classes should not depend on sub-classes. Both should be abstracted with interfaces.
- Interfaces should not depend on details,
  but details should depend on the interfaces.
- This principle is mainly applied on classes, but they are also valid for modules.

Goal: Improve the reusability of all components of a program.

Layer-coupling

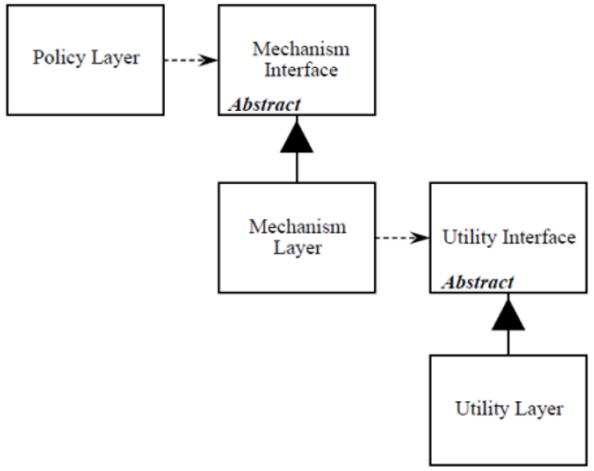
Problem



- Every change in **Utility** forces recompilation up-stack.
- This hinders reuse & agile evolution.

These dependencies violate the Dependency Inversion Principle!

Solution



- Layers separated using interfaces
- Interfaces defined based on requirements of objects on the upper-level.
- Classes of the levels below implement these interfaces.

OrderService (C#, without DIP (tight coupling))

```
public class OrderService
{
    private readonly SqlOrderRepository repo = new SqlOrderRepository(); // concrete

    public void PlaceOrder(Order order)
    {
        repo.Save(order);
    }
}
```

`OrderService` cannot run with an in-memory repo for tests.

OrderService (C#, after DIP + DI)

```
public interface IOrderRepository
{
    void Save(Order order);
}

public class DatabaseOrderRepository : IOrderRepository { ... }


public class OrderService
{
    private readonly IOrderRepository repo;
    public OrderService(IOrderRepository repo) => this.repo = repo;

    public void PlaceOrder(Order order) => repo.Save(order);
}
```

High-level code depends on abstraction only.

AccountService (Java, after DIP)

```java
public interface NotificationSender {
    void send(String message);
}

public class EmailSender implements NotificationSender { ... }


@Service
public class AccountService {
    private final NotificationSender sender;
    public AccountService(NotificationSender sender) { this.sender = sender; }
}
```

Swap `EmailSender` vs `SmsSender` without touching `AccountService`.

DIP - Best Practices

- Define abstractions **in the consumer module** (not in the provider).
- Let policies depend on abstractions; push details down.
- Avoid `new` inside high-level code; use factories/DI.
- Combine with **ISP** to keep abstractions minimal.
- Watch for *leaky abstractions* (methods only needed by one implementation).

Conclusion

- Reduce dependencies between different levels.
- Upper levels should not depend on lower levels.

- Dependency Inversion is a principle.
- Dependency Injection is a programming technique to enforce Dependency Inversion.

Dependency Inversion != Dependency Injection

Further Reading

- Baeldung: The Dependency Inversion Principle in Java



Dependency Injection

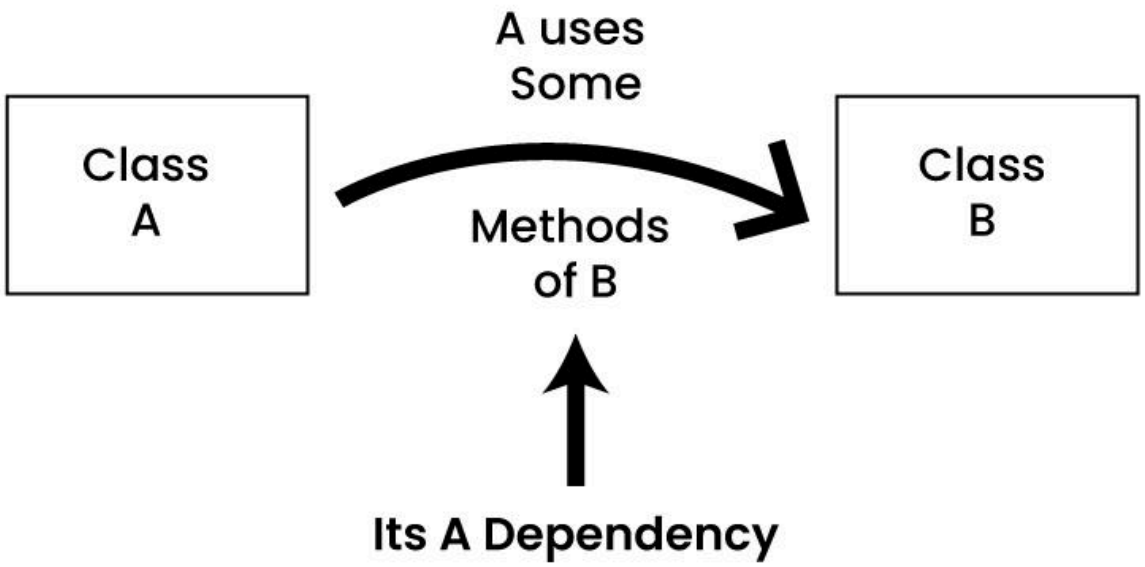
FH University of Applied Sciences
TECHNIKUM WIEN

Dependency Injection vs Dependency Inversion

- Dependency Inversion is a principle.
- Dependency Injection is a programming technique to enforce Dependency Inversion.

	Principle (DIP)	Technique (DI)
What	Design rule	Concrete pattern to supply deps
Goal	Direction of dependency	Object creation & lifetime
Relation	DI realises DIP in code	Can be manual or via container

Dependency Injection



- Dependencies will be applied at runtime.
- A central „service“-object manages the creation and availability of objects.
- When an object of type A needs an object of type B, so it does not need to instantiate it himself.
  A central „service“-object, which implements the management, will provide the instance of type B.

DI Constructor Injection

```
public class Dependent {
    private SomeThing someThing;
    public Dependent (final SomeThing someThing) {
        this.someThing = someThing
    }
}

public class Injector {
    public void handleDependency () {
        final SomeThing someThing = /*create object here*/
        final Dependent dependent = new Dependent(someThing);
    }
}
```

DI Constructor Injection (C#)

```
builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();

...

public class CheckoutController
{
    private readonly IOrderRepository repo;
    public CheckoutController(IOrderRepository repo) => this.repo = repo;
}
```

ASP.NET Core's built-in container resolves and disposes dependencies

Popular DI frameworks for C#

- **.NET** – Built-in (Microsoft.Extensions.*)
- **Autofac**
- **Ninject**

DI Constructor Injection (Java)

```java
@Component
class SqlOrderRepository implements OrderRepository { … }

@Service
@RequiredArgsConstructor  // Lombok
class CheckoutService {
    private final OrderRepository repo;   // injected
}
```

The Spring container autowires dependencies at runtime.

Popular DI frameworks in Java

- Spring / Spring Boot
- Jakarta CDI
- … Guice, Dagger 2, Micronaut, Quarkus

DI Interface Injection

```
public interface Injectable {
    public void inject (final SomeThing someThing)
}

public class Dependent implements Injectable {
    public final SomeThing someThing;
    public void inject (final SomeThing someThing) {
        this.someThing = someThing;
    }
}

public class Injector {
    public void handleDependency() {
        final SomeThing someThing = /* create object here */
        final Injectable injectable = /* create object here */
        injectable.inject(someThing);
    }
}
```

DI Setter Injection

```
public class Dependent {
     private SomeThing someThing;
     public void setDependency (final Dependency dependency) {
          this.someThing = someThing;
     }
}

public class Injector {
     public void inject() {
          final Dependent dependent = /* create object here */
          final SomeThing someThing = /* create object here */
          dependent.setDependency(someThing);
     }
}
```

Anti-Patterns & Pitfalls

- Service Locator hides dependencies — code looks simpler but coupling remains
- Over-injecting tiny services → configuration bloat.
- Multiple containers → duplicated lifetimes.
- Forgetting to register **scoped** vs **singleton** correctly → memory leaks or state bleed.

Conclusion

- Reduce dependencies between different levels.
- Upper levels should not depend on lower levels.
- Use Dependency Injection to realise DIP without manual factories.
- Modern frameworks (.NET, Spring, etc.) make DI almost free — leverage them!

Further Reading

- Microsoft Learn: Dependency injection in ASP.NET Core
- Spring Docs: Dependency Injection